



# Softmax and gradient descent
# October 5th, 2026

These slides are based on material from Professor Jonathan Kao, who also developed it based on other freely available sources including from Stanford and other resources, whom we acknowledge.

# Announcements

- HW #1 is due Friday October 9, 11.59pm uploaded to Gradescope. **Note that even 1 minute late is considered a day late**

- **Please read the rules handout carefully including for late HWs etc.**

- To submit your Jupyter Notebook, print the notebook to a pdf with your solutions and plots filled in. The graders will grade your work from these notebooks. In the future, you will also have to print out code from any .py files that were modified.

- Discussion sessions held on Fridays 11am, will be recorded and live zoom link also is available.

- Lecture on Wednesday, October 7th converted into review for class background

  - Makeup lecture on Friday Oct 2 — online recording on Brainlearn

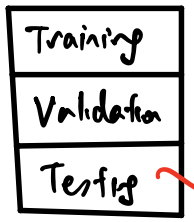
# Review of previous lecture

- Model generalization: what we care about is performance on unseen data
  - Notions of overfitting and underfitting

"low training error, high error on unseen data" | "high training error" | how to measure these

  - Cross-validation: Divide data into training, validation and testing parts.

Data: $\mathcal{D} = \{(\underline{x}^{(1)}, y^{(1)}), \ldots, (\underline{x}^{(N)}, y^{(N)})\}$ divide $\mathcal{D}$ =

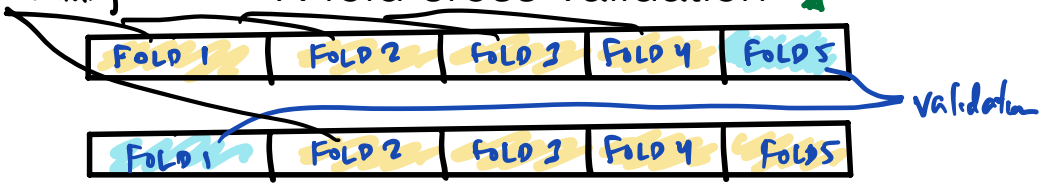



Testing: untouched till the end

ASSUMPTION: data comes from same distribution

if test data is not same dist: out-of-distribution

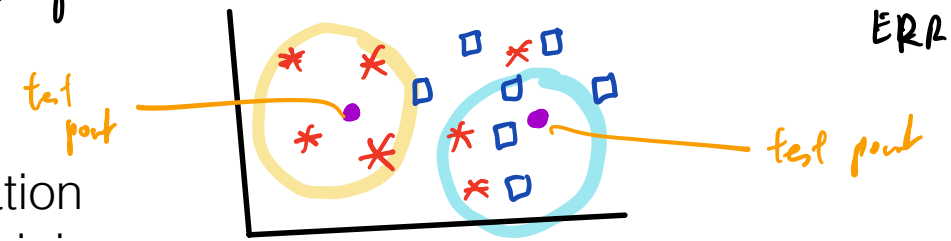
  - K-fold cross-validation




Hyperparameter selection through validation

- Image classification: challenges of viewpoint, zoom, background, illumination, occlusion inter-class variations etc.

↳ need to develop methods & representations that are robust to such variations




Cover-Hart '68

$ERR_{NC} \leq 2\,ERR_{opt} + O\left(\frac{1}{N^{\frac{1}{d}}}\right)$

# of samples

d dimension

- K-nearest neighbor classification
  - Pros: simple without training
  - Cons: memory, inference time compute, curse of dimensionality
    - Choice of distance harder to do to be robust.

# Classifiers based on linear classification

Perhaps a better way would be to develop a "score" for an image coming from each class, and then pick the class that achieves the highest score.

Linear classifiers are a major building block for neural networks. In particular, a common neural network layer (a multi-layer perceptron) comprises an affine transformation, followed by a nonlinear function.

The softmax classifier is the most common classifier at the end of a neural network.

- Builds towards Neural networks (NN) → softmax is usually last layer of NN

Input → $x^{(i)}$ devise a score $s_j(\underline{x}^{(i)})$, $j^{th}$ class

$\underline{z} \in \mathbb{R}^C$ $\quad \underline{z}(\underline{x}^{(i)}) = \begin{bmatrix} s_1(\underline{x}^{(i)}) \\ \vdots \\ s_C(\underline{x}^{(i)}) \end{bmatrix}$

$C$ = # of classes

score for class 1 cat

score of airplane etc

pick the largest score for your classifier

# Classifiers based on linear classification

Example 2: Consider a matrix $\mathbf{W}$, defined as:

$$\mathbf{W} = \begin{bmatrix} \mathbf{w}_1^T \\ \vdots \\ \mathbf{w}_c^T \end{bmatrix}$$

($c$ rows, $n$ columns)

Where $\mathbf{W} \in \mathbb{R}^{c \times n}$

bias term

$$\underline{z} = \underline{W}\,\underline{x} + \underline{b}$$

input

$$\underline{z} = \begin{bmatrix} \underline{w}_1^T \underline{x} + \underline{b} \\ \vdots \\ \underline{w}_c^T \underline{x} + \underline{b} \end{bmatrix}$$

$\underline{z}$: "score"

Question: When is a particular class give larger $\underline{z}$ component

ie what makes the "score" larger?

Let $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, where $\mathbf{b}$ is a vector of bias terms. Then $\mathbf{z} \in \mathbb{R}^c$ is a vector of scores with its $i$-th element corresponding to the score of $\mathbf{x}$ being in class $i$. The chosen class corresponds to the index of the highest score in $\mathbf{z}$.

When $\underline{W}_i$ is nearly colinear with $\underline{x}$

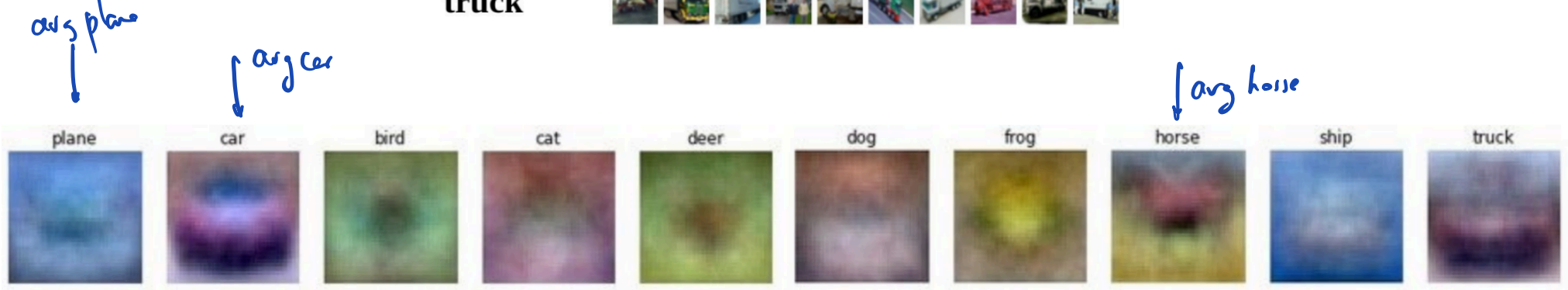
$$\underline{W}_i^T \underline{x} = \langle \underline{W}_i, \underline{x} \rangle = \|\underline{W}_i\| \cdot \|\underline{x}\| \cos\theta_i$$

$\theta_i \approx 0$

# Classifiers based on linear classification

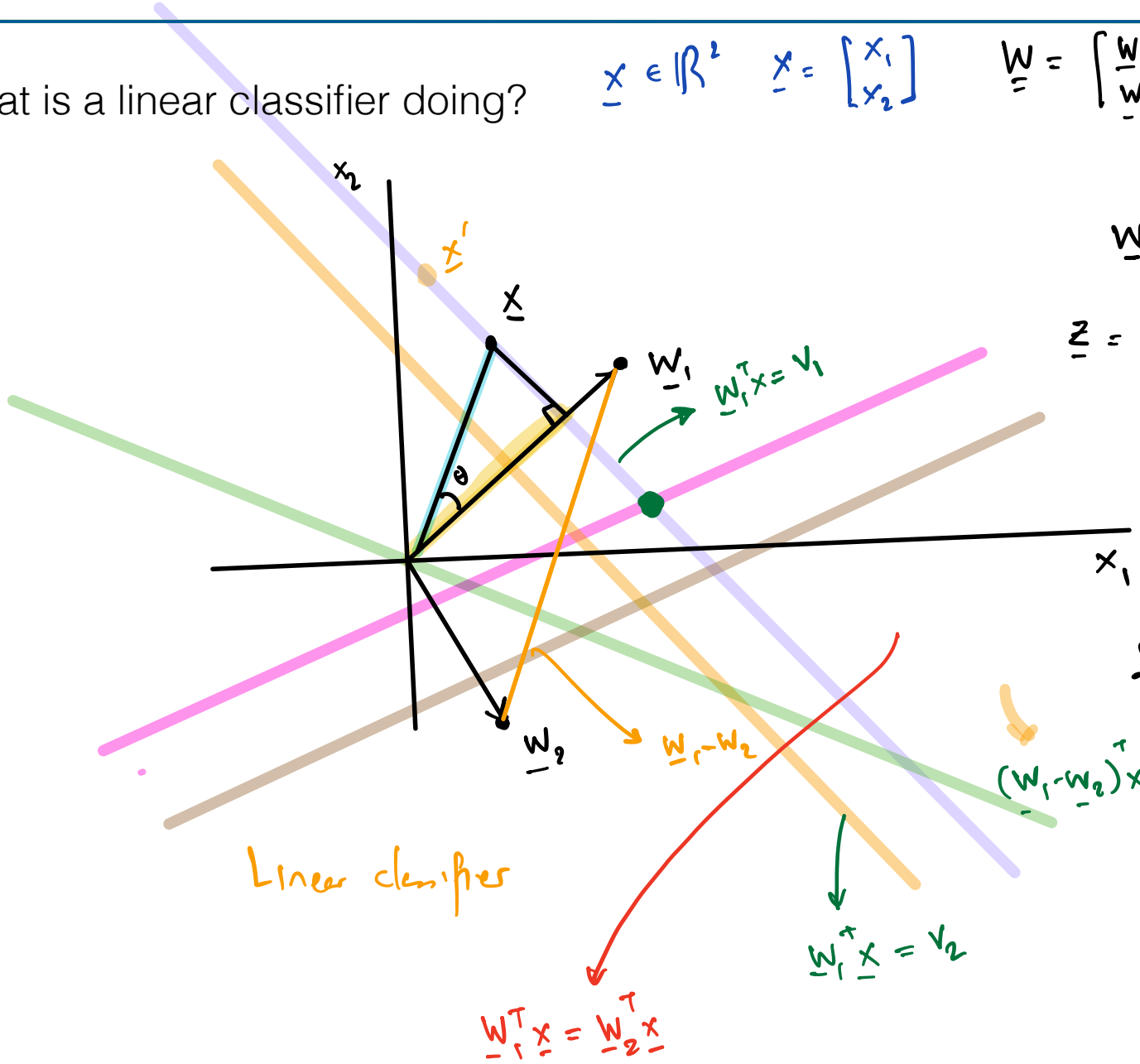
Each row of **W** can be thought of as a template.



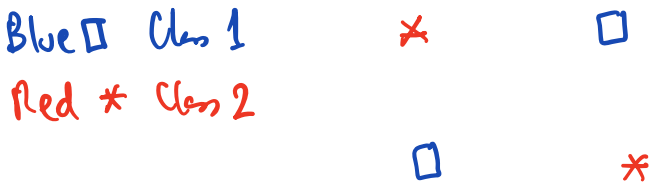
# Classifiers based on linear classification

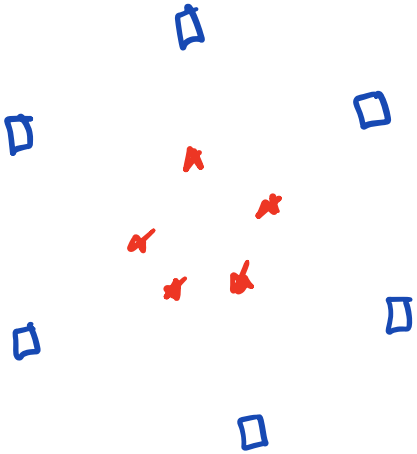
What is a linear classifier doing?



# Classifiers based on linear classification

Where might linear classifiers fail?



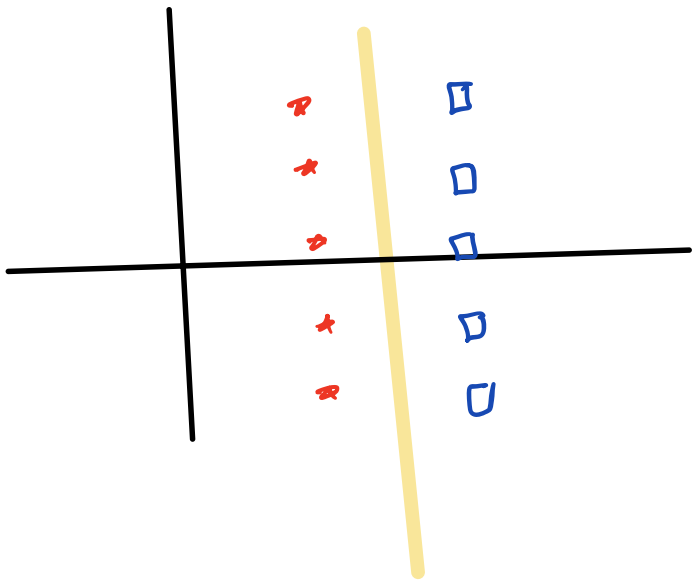



Issue: Linear classifiers are not able to classify correctly

Solution: To represent the data such that a linear classifier can work

→ NN end up doing

polar coordinates transformation





# Classifiers based on linear classification

Next, how do we take the scores we receive (which are analog in value) and turn them into an appropriate **loss function** for us to optimize, so we can learn **W** and **b** appropriately?

- How to choose $\underline{w}_1, \underline{w}_2, \ldots$ etc
- For example earlier for regression we used a quadratic loss function
  - ↳ what loss function should we use?

# Other types of optimization

We've talked about examples where we want to *minimize* a mean-square error or distance metric.

Another metric that we may want to maximize is the *probability of having observed the data*. In this framework, the data is modeled to have some distribution with parameters. We choose the parameters to maximize the probability of having observed our training data.

Simple example  dataset : coin tosses  Goal: Find the bias of a coin

Data : $(\mathcal{D})$ H T H H T T H T  Model: $x^{(i)} = \begin{cases} H & \text{wp } \theta \\ T & \text{wp } 1-\theta \end{cases}$

- Independent cointosses

Likelihood: Probability of seeing the data given a model

$\theta \rightarrow$ what we want to estimate

e.g. If $\theta = 0.75$ what is the probability of $\mathcal{D}$

$P(\mathcal{D}) = (0.75)(0.25)(0.75)(0.75)(0.25)(0.25)(0.75)(0.25) = (0.75)^4(0.25)^4$

for general $\theta$  $P(\mathcal{D}) = \theta^{\#\text{of H}}(1-\theta)^{\#\text{of tails}}$

Maximum likelihood principle

Choose $\theta$ that maximizes the probability of observing the data.

# Maximum-likelihood introduction

$$\mathcal{L}(\theta) = \mathbb{P}(\mathcal{D}|\theta) = \theta^{\#\text{ of H}}(1-\theta)^{\#\text{ of tails}}$$

$n_H$ = # of H

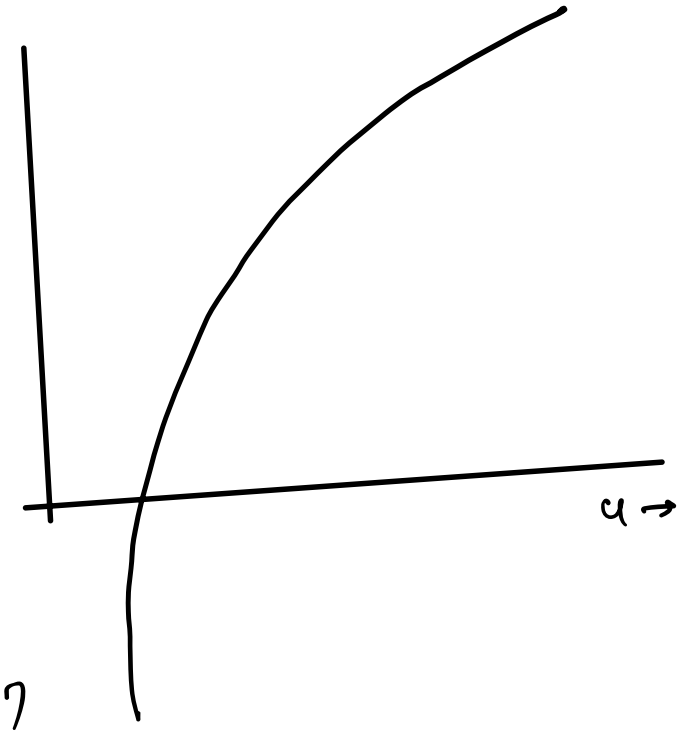
$n_T$ = # of T

$$\arg\max_\theta \mathcal{L}(\theta) = \arg\max_\theta \theta^{n_H}\cdot(1-\theta)^{n_T}$$

log likelihood

$$\log \mathcal{L}(\theta) = \log \theta^{n_H}\cdot(1-\theta)^{n_T} = n_H \log\theta + n_T \log(1-\theta)$$

If we $\arg\max \mathcal{L}(\theta) = \arg\max \log \mathcal{L}(\theta)$




$$\arg\max_\theta \; n_H \log\theta + n_T \log(1-\theta)$$

How? find $\frac{\partial}{\partial\theta}\left[\log \mathcal{L}(\theta)\right] = 0$ to find $\theta$.

Exercise $\mathcal{D}$ = H T H H T T T H what is optimal $\theta$?

↳ $\theta = \frac{1}{2}$

# Chain rule for probability

$\mathbb{P}(A=a) = p_A(a) = p(a)$ → for simplicity of notation

$\mathbb{P}(B=b) = p_B(b) = p(b)$

$\mathbb{P}(A=a, B=b) = p_{A,B}(a,b) = p(a,b)$

Chain rule

- $p(a,b) = p(a) \cdot p(b|a) = p(b) \cdot p(a|b)$
- $p(a,b,c) = p(a) \cdot p(b|a) \cdot p(c|a,b)$

Questions

- $p(b,c|d,e) = \dfrac{?}{p(d) \cdot p(e|d)}$

  ? → $p(b,c,d,e)$

  $p(b,c|d,e) \cdot p(d,e) = p(b,c,d,e)$

  $p(d) \cdot p(e|d) \rightarrow p(d,e)$

- $p(d,e)$ ? $= \left\{ \dfrac{p(a,b,c,d,e)}{p(a|b,c,d,e)} \right.$

  $p(a,b,c,d,e) \cdot p(a|b,c,d,e)$

  $p(b,c|d,e)$ → $p(b,c,d,e)$

$\Rightarrow$ ? $= \dfrac{p(b,c,d,e)}{p(d,e)}$

# Turn the scores into a probability

A first thought is to turn the scores into probabilities.

## Softmax function

There are several instances when the scores should be normalized. This occurs, for example, in instances where the scores should be interpreted as probabilities. In this scenario, it is appropriate to apply the *softmax* function to the scores.

The softmax function transforms the class score, $\mathrm{softmax}_i(\mathbf{x})$, so that:

Connections to IT physics

$$\mathrm{softmax}_i(\mathbf{x}) = \frac{e^{a_i(\mathbf{x})}}{\sum_{j=1}^{c} e^{a_j(\mathbf{x})}}$$

→ a number $[0,1]$

$\sum_i \mathrm{softmax}_i(x) = 1$

scores we had from before

for $a_i(\mathbf{x}) = \mathbf{w}_i^T \mathbf{x} + b_i$ and $c$ being the number of classes.

Reinterpret: Softmax as the probability of a class given the input.

Converting scores $z_i = a_i(\underline{x}) = \underline{w}_i^T x + b_i$ into numbers $[0,1]$ and actually probability.

# Softmax classifier

$$\text{softmax}_i(\mathbf{x}) = \frac{e^{a_i(\mathbf{x})}}{\sum_{j=1}^{c} e^{a_j(\mathbf{x})}}$$

$\in [0,1]$ and adds up to 1.

for $a_i(\mathbf{x}) = \mathbf{w}_i^T \mathbf{x} + b_i$ and $c$ being the number of classes.

If we let $\theta = \{\mathbf{w}_j, b_j\}_{j=1,\ldots,c}$, then $\text{softmax}_i(\mathbf{x})$ can be interpreted as the probability that $\mathbf{x}$ belongs to class $j$, that is

$$\text{Prob}(y^{(i)} = j \,|\, \mathbf{x}^{(i)}, \theta) = \text{softmax}_j(\mathbf{x}^{(i)}).$$

$\mathcal{D} = \{ (x^{(1)}, y^{(1)}) \ldots (x^{(N)}, y^{(N)}) \}$

$\text{Softmax}_j(x^{(i)}) = \mathbb{P}(\text{image } x^{(i)} \text{ belongs to the class } j)$

## Softmax classifier

$\mathcal{D} = \{(\underline{x}^{(1)}, y^{(1)}), \ldots, (\underline{x}^{(N)}, y^{(N)})\}$

Although we know the softmax function, how do we specify the *objective* to be optimized with respect to $\theta$?

One intuitive heuristic is that we should choose the parameters, $\theta$, so as to maximize the likelihood of having seen the data. Assuming the samples, $(\mathbf{x}^{(1)}, y^{(1)}), \ldots, (\mathbf{x}^{(m)}, y^{(m)})$ are iid, this corresponds to maximizing:

$$P(\mathcal{D}|\theta) = p(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}, y^{(1)}, \ldots, y^{(m)}|\theta) \overset{\text{independence}}{=} \prod_{i=1}^{m} p(\mathbf{x}^{(i)}, y^{(i)}|\theta)$$

ASSUMPTION: Data is independent across components

If A, B independent and identically distributed as p

$p(A,B) = p(A) \cdot p(B)$

$$\overset{\text{chain rule}}{=} \prod_{i=1}^{m} p(\mathbf{x}^{(i)}|\theta) p(y^{(i)}|\mathbf{x}^{(i)}, \theta)$$

$p(x^{(i)}, y^{(i)}|\theta)$

Maximum likelihood principle: Find $\theta$ that maximizes the probability of seeing $\mathcal{D}$.

$P(\mathcal{D}|\theta) = \prod_{i=1}^{m} p(x^{(i)}|\theta) \cdot p(y^{(i)}|x^{(i)}, \theta)$, like before we will take log to convert product to sums.

# Softmax classifier

Question does $p(x^{(i)}|\theta)$ depend on $\theta$?

$$\arg\max_\theta \prod_{i=1}^m p(\mathbf{x}^{(i)}|\theta)p(y^{(i)}|\mathbf{x}^{(i)},\theta) = \arg\max_\theta \prod_{i=1}^m p(y^{(i)}|\mathbf{x}^{(i)},\theta)$$

does not depend on $\theta$

$$\underset{\theta}{\operatorname{argmax}} \sum_{i=1}^m \log p(y^{(i)}|x^{(i)},\theta)$$

$\hookrightarrow$ softmax$_{y^{(i)}}(x^{(i)})$

$$= \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^m \log\left[\text{softmax}_{y^{(i)}}(x^{(i)})\right] = \underset{\theta}{\operatorname{argmax}} \left[\log \frac{e^{W_{y^{(i)}}^T x^{(i)} + b_{y^{(i)}}}}{\sum_{j=1}^c e^{[W_j^T x^{(i)} + b_j]}}\right]$$

$\log(a/b) = \log a - \log b$

$$= \underset{\theta}{\operatorname{argmax}} \left[ a_{y^{(i)}}(x^{(i)}) - \log \sum_{j=1}^c a_j(x^{(i)}) \right]$$

$\log e^{(\cdot)} = (\cdot)$

$W_{y^{(i)}}^T x^{(i)} + b_{y^{(i)}} = a_{y^{(i)}}(x^{(i)})$

$$a_i(\underline{x}^{(i)}) = \underline{W}_i^T \underline{x}^{(i)} + b_i$$

$$a_j(\underline{x}^{(i)}) = \underline{W}_j^T \underline{x}^{(i)} + b_j$$

# Softmax classifier

Note, we haven't figured out yet how to get the optimal parameters. (That'll be later.)

The negative log-likliehood loss function is commonly called the "cross-entropy loss":

$$\arg\min_{\theta} \sum_{i=1}^{m} \left( \log \sum_{j=1}^{c} e^{a_j(\mathbf{x})} - a_{y^{(i)}}(\mathbf{x}^{(i)}) \right)$$

where $z^{(i)}$ is a one-hot representation of the class $y^{(i)}$ so that $z_j^{(i)} = 1$ if $y^{(i)} = j$, and

one-hot

$$z_j^{(i)} = \begin{cases} 1 & \text{if } j = y^{(i)} \\ 0 & \text{else} \end{cases}$$

$$z^{(i)} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \rightarrow y^{(i)} = 3$$

one hot representation

$$\mathcal{L} = -\sum_{i=1}^{m} \left( \sum_{j=1}^{c} z_j^{(i)} \log p_j^{(i)} \right)$$

$\{0, 1\} \rightarrow 1$ if $j = y^{(i)}$

only one element of the sum is picked the element being $j = y^{(i)}$

$$p_j^{(i)} = \Pr\left(y^{(i)} = j | \mathbf{x}^{(i)}, \theta\right)$$

$\text{softmax}_j(x^{(i)})$

$$\sum_{i=1}^{m} \log P_{y^{(i)}}^{(i)}$$



# Softmax classifier

$$\arg\min_{\theta} \sum_{i=1}^{m} \left( \log \sum_{j=1}^{c} e^{a_j(\mathbf{x})} - a_{y^{(i)}}(\mathbf{x}^{(i)}) \right)$$

VS

$$\mathcal{L} = -\sum_{i=1}^{m} \left( \sum_{j=1}^{c} z_j^{(i)} \log p_j^{(i)} \right)$$

$$= -\sum_{i=1}^{m} \left( \sum_{j=1}^{c} z_j^{(i)} \log \Pr\left(y^{(i)} = j | \mathbf{x}^{(i)}, \theta\right) \right)$$